



SmartPot

DOCUMENTACIÓN DEL COMPONENTE

SmartPot-API

El centro de la plataforma SmartPot

REST con JWT, puente MQTT con el broker, cultivos reales y virtuales, agente de automatización con el asistente de IA, canales de notificación, seguridad, configuración y operación de la API.



DOCUMENTO
SmartPot-API

VERSIÓN
1.0 · septiembre 2026

EQUIPO
SmartPotTech

PROYECTO
smartpot.app



DOCUMENTO

Ficha del documento

CAMPO	VALOR
Proyecto	SmartPot · smartpot.app
Componente	SmartPot-API
Versión	1.0 · septiembre 2026
Alcance	Dominios de la API, cultivos reales y virtuales, flujo de lecturas y comandos, seguridad, configuración, pruebas y operación
Documentación de la plataforma	Documentación técnica , recorrido del proyecto , ciclo de vida y diagramas generales
Mantenimiento	Se genera desde docs/ de este repositorio con las herramientas de .github/docs/tools ; se actualiza con cada cambio del componente

CONTENIDO

Contenido

1. Propósito
2. Arquitectura del componente
3. Cultivos reales y virtuales
4. De la lectura a la orden
5. Seguridad de cada petición
6. Contrato
7. Configuración
8. Pruebas
9. Operación



PARTE I

El componente

SECCIÓN 01

Propósito

En palabras simples

La API es la única puerta de SmartPot. La PWA le habla por HTTPS; los dispositivos de los cultivos, por MQTT a través del broker. La API guarda las lecturas, le pregunta al asistente de IA cómo va cada cultivo, manda las órdenes a los actuadores, administra las cuentas MQTT de cada cultivo y avisa por la PWA y por Telegram. Todo lo que ve una persona pasa por aquí, siempre con su sesión y solo sobre sus propios cultivos.

RESPONSABILIDAD	CÓMO
Cuentas y sesión	Registro, ingreso con JWT, recuperación de contraseña por correo, perfil y borrado de la cuenta
Cultivos	Reales o virtuales (se elige al crear y no cambia), seis especies, cuatro formas y modo automático
Dispositivos	Cuenta MQTT por cultivo con clave de 192 bits cifrada con AES-256-GCM; aprovisionamiento en Mosquitto
Lecturas	Telemetría MQTT validada contra el rango físico de cada sensor, historial, resumen y CSV
Órdenes	Comandos con QoS 1, confirmación del dispositivo, vencimiento y órdenes en bloque
Asistente	Evaluación con SmartPot-AI, agente de automatización y envío de lecturas reales para el aprendizaje
Simulación	Configuración de los cultivos virtuales y proxy al simulador interno
Avisos	Notificaciones en la PWA y en Telegram, con canales intercambiables



SECCIÓN 02

Arquitectura del componente

Cada dominio sigue las mismas capas (`controller`, `service`, `repository`, `mapper` y `model`). Los dominios se hablan con servicios y eventos de Spring (`ReadingRecordedEvent`, `CropDeletedEvent`, `DeviceKeyRotatedEvent`, `NotificationCreatedEvent`), así un cambio en uno no arrastra a los demás.

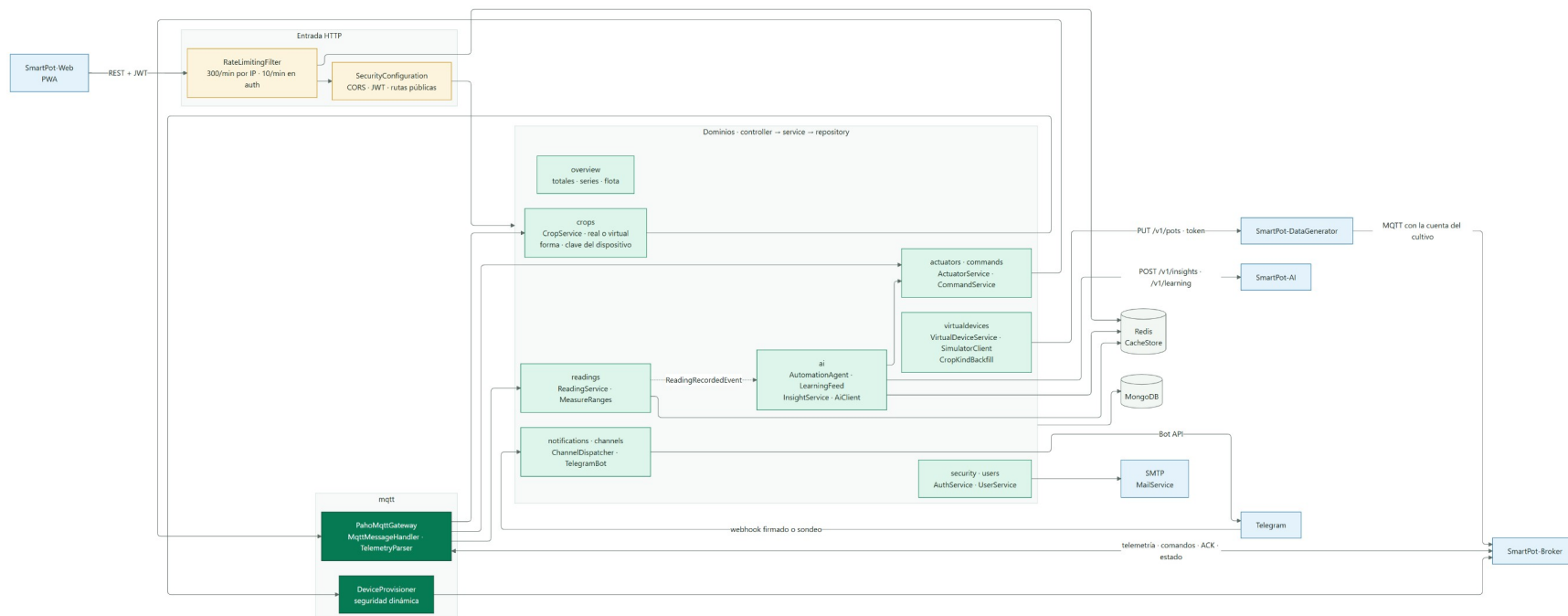


Figura 1 · SmartPot-API por dentro



PAQUETE	QUÉ CONTIENE
security, users	Filtros, JWT, límite de peticiones, AES-GCM, autenticación y perfil
crops	Cultivos, tipo y forma, modo automático y credenciales del dispositivo
readings	Lecturas, rangos físicos, resumen y exportación
actuators, commands	Actuadores de cada cultivo y ciclo de vida de las órdenes
mqtt	Cliente Paho, tópicos v1, parser de telemetría y aprovisionamiento en el broker
ai	Cliente de la IA, evaluación, agente de automatización y LearningFeed
overview	Panel general: totales, series con <code>\$dateTrunc</code> y análisis de flota
notifications, channels	Alertas y canales externos (<code>NotificationChannel</code> , hoy Telegram)
virtualdevices	Simulación de los cultivos virtuales, reconciliación y completado de cultivos anteriores
cache, mail, health, config	Redis con respaldo en memoria, correo, <code>/health</code> y configuración

SECCIÓN 03

Cultivos reales y virtuales

Al crear un cultivo se elige, una sola vez, de dónde vienen sus lecturas. Los dos tipos tienen cuenta en el broker; lo que cambia es quién la usa.

REGLA	REAL	VIRTUAL
Quién publica	Un ESP32 con el firmware, físico o simulado en Wokwi	SmartPot-DataGenerator
Credenciales	Se entregan una vez y se pueden rotar	No se entregan: la API se las pasa al simulador por la red interna
Actuadores al nacer	Bomba, luz de cultivo y ventilador	Los seis
Rutas propias	<code>/device</code> y <code>/device/key</code>	<code>/virtual-device</code> (PUT cambia o reanuda, DELETE pausa)
Aprendizaje continuo	Sus lecturas entrenan los modelos	Sus lecturas no se envían
Límite	20 cultivos por cuenta en total	Hasta 5 por cuenta

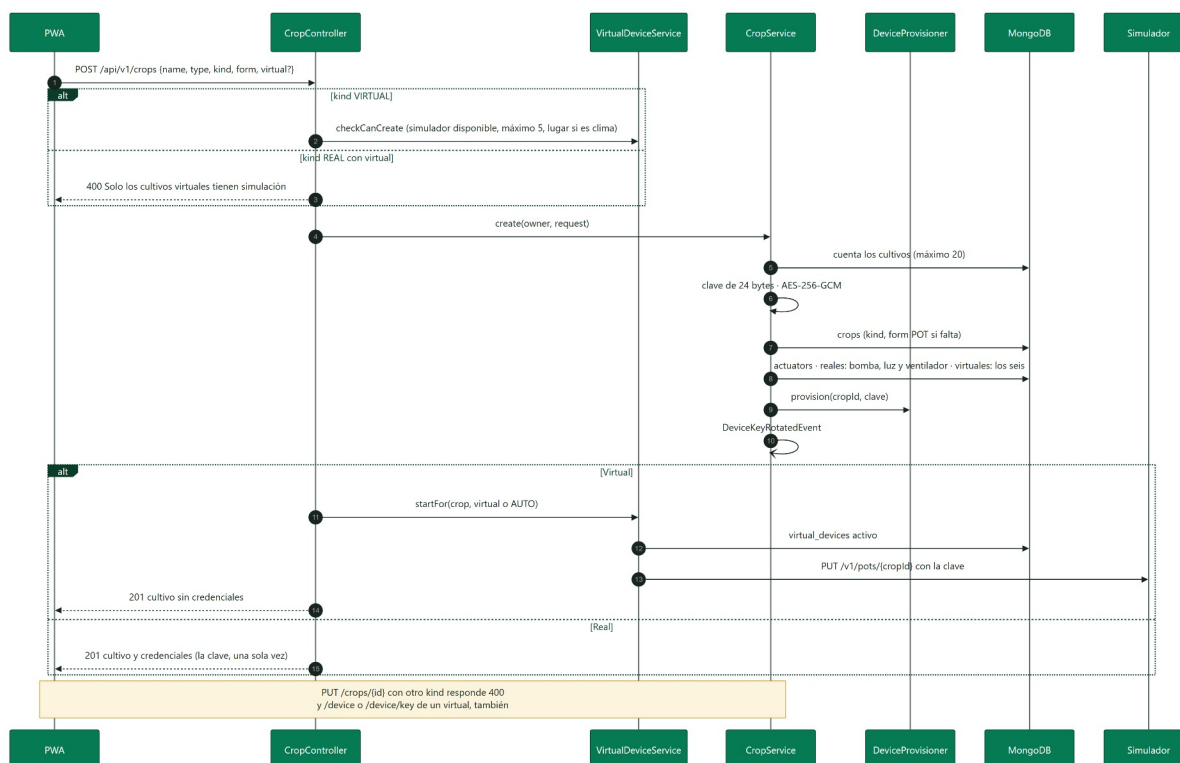


Figura 2 · Crear un cultivo real o virtual

La forma (POT, NFT, TOWER, RAFT) solo decide cómo lo dibuja la PWA y se puede editar. Los cultivos creados antes de existir el tipo se completan al arrancar con **CropKindBackfill**: los que tenían simulación pasan a virtuales, el resto a reales, y sin forma quedan como maceta.



PARTE II

Funcionamiento

SECCIÓN 04

De la lectura a la orden

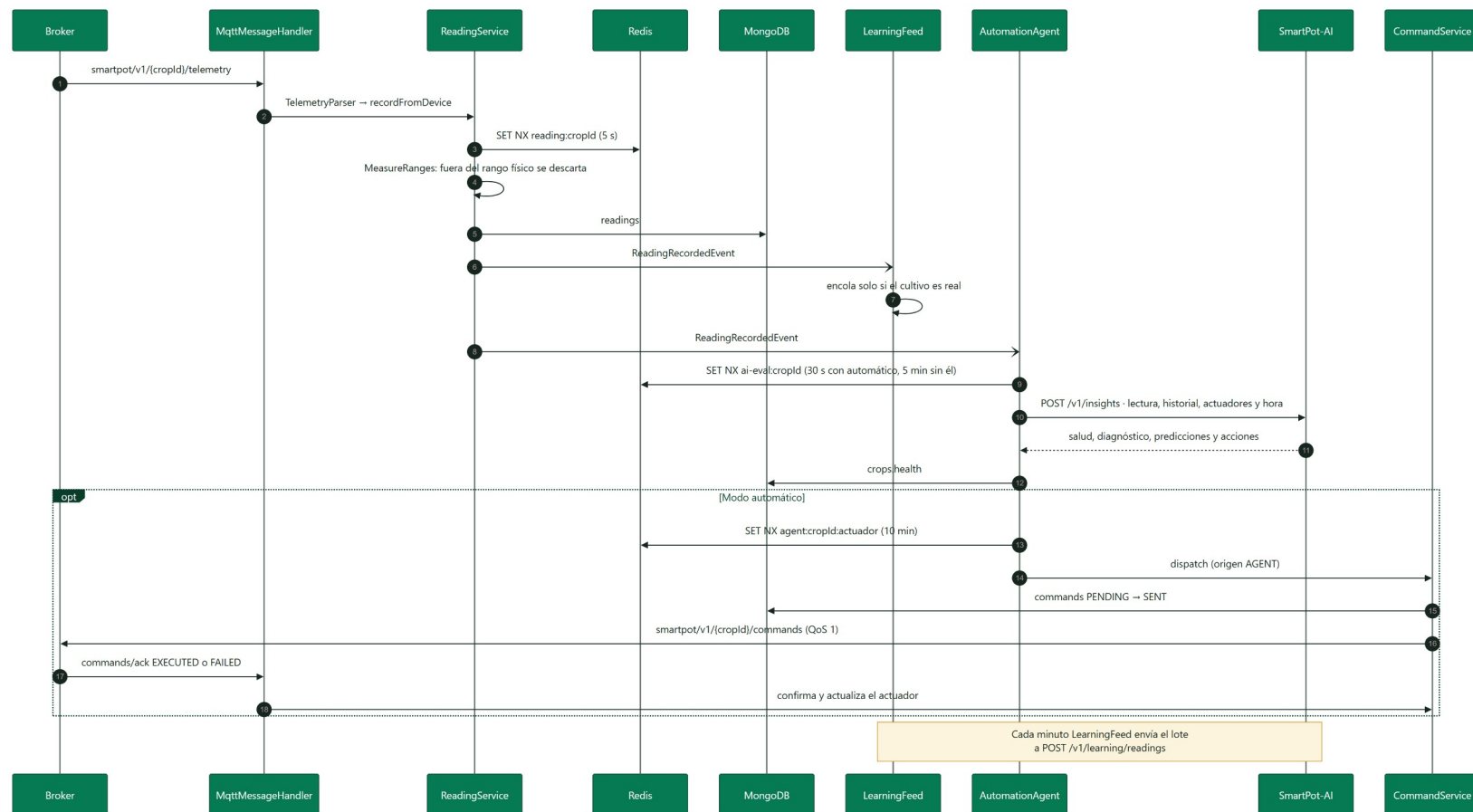


Figura 3 · De la lectura a la orden automática



REGLA	VALOR
Lecturas por cultivo	Máximo una cada 5 s; las demás se descartan
Validación	Cada variable dentro del rango físico del sensor; si no, se descarta la lectura completa
Evaluación	Como máximo cada 30 s con modo automático y cada 5 min sin él
Enfriamiento del agente	10 minutos por actuador
Vencimiento de un comando	2 minutos sin ACK → EXPIRED y aviso
Lote de aprendizaje	Cada minuto, hasta 1000 lecturas por envío; la cola guarda 20 000

SECCIÓN 05

Seguridad de cada petición

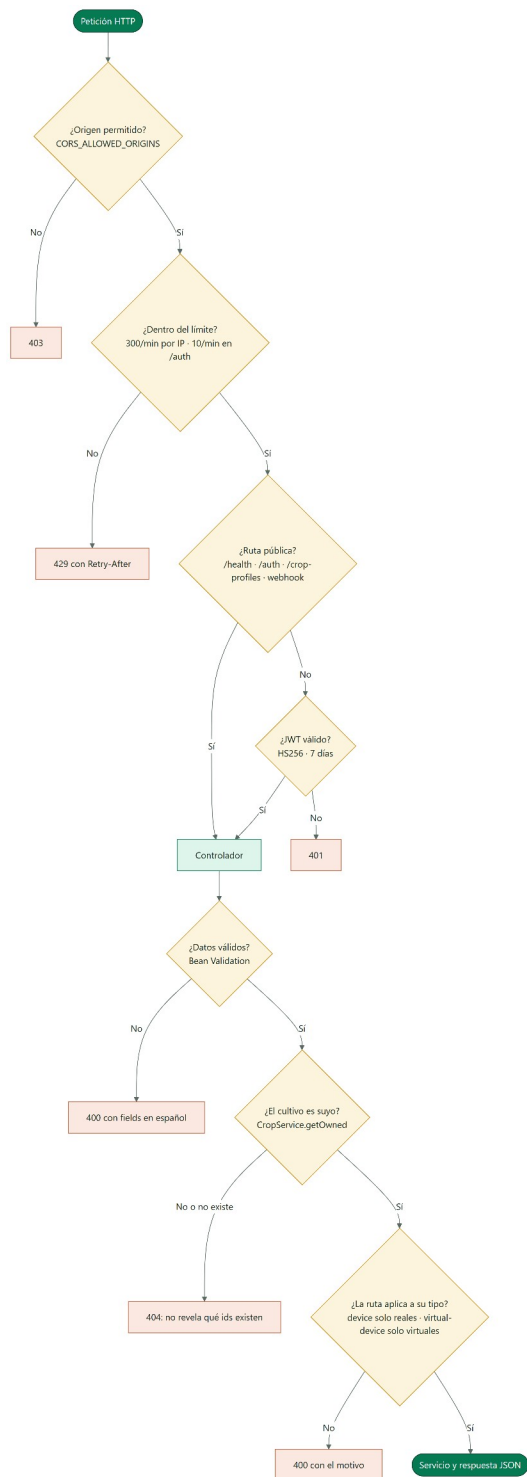


Figura 4 · Qué revisa la API en cada petición



CONTROL	DETALLE
Contraseñas	8 caracteres a 72 bytes con mayúscula, minúscula y número; BCrypt de costo 12
Recuperación	Responde 202 aunque el correo no exista; token de 30 minutos guardado como SHA-256
Claves de los dispositivos	24 bytes aleatorios, cifrados con AES-256-GCM; la llave es SMARTPOT_AES_KEY
Servicios internos	La IA y el simulador se llaman con token de servicio por la red interna
Telegram	Webhook con secreto comparado en tiempo constante; códigos de vinculación de un solo uso

SECCIÓN 06

Contrato

La documentación interactiva vive en [/docs](#) y el esquema en [/v3/api-docs](#); el [README](#) resume todas las rutas y la [documentación técnica](#) detalla el contrato MQTT v1.

GRUPO	RUTAS
Público	/health, /api/v1/auth/*, /api/v1/crop-profiles
Cultivos	/api/v1/crops, /crops/{id}, /crops/automation, /crops/{id}/device
Datos del cultivo	/readings, /actuators, /commands, /insights, /virtual-device
Cuenta	/api/v1/users/me, /notifications, /channels, /overview, /commands, /ai/learning

PARTE III

Operación

SECCIÓN 07

Configuración

La API lee `.env` desde la carpeta del proyecto; `.env.example` lista todas las variables.

GRUPO	VARIABLES
Servidor	PORT, LOG_LEVEL, WEB_BASE_URL, PUBLIC_API_URL, CORS_ALLOWED_ORIGINS, API_DOCS_ENABLED
Secretos	SMARTPOT_JWT_SECRET (32 caracteres o más), SMARTPOT_AES_KEY (AES-256 en Base64), JWT_EXPIRATION
Datos	MONGODB_URI, REDIS_HOST, REDIS_PORT, REDIS_PASSWORD
Correo	MAIL_ENABLED, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM
MQTT	MQTT_ENABLED, MQTT_BROKER_URI, MQTT_ADMIN_USERNAME, MQTT_ADMIN_PASSWORD, MQTT_PUBLIC_*, MQTT_WEBSOCKET_URL
IA	AI_ENABLED, AI_BASE_URL, SMARTPOT_AI_TOKEN, AI_EVALUATION_INTERVAL, AI_AUTOMATION_COOLDOWN, AI_LEARNING_*
Telegram	TELEGRAM_BOT_TOKEN, TELEGRAM_BOT_USERNAME, TELEGRAM_MODE (polling o webhook), TELEGRAM_WEBHOOK_SECRET
Simulador	SIMULATOR_ENABLED, SIMULATOR_BASE_URL, SIMULATOR_TOKEN (sin él no se crean cultivos virtuales)

LLAVE AES

Si SMARTPOT_AES_KEY se pierde o cambia, las cuentas MQTT no se pueden reconstruir: cada dueño debe rotar la clave de sus cultivos reales.

SECCIÓN 08

Pruebas

`./mvnw verify` corre 118 pruebas con JUnit y Mockito: cifrado, JWT, contraseñas, tópicos y telemetría MQTT, aprovisionamiento, comandos, cultivos con su tipo fijo y su forma, el agente, el envío de lecturas para el aprendizaje (solo reales), canales y bot de Telegram, la simulación de los virtuales (dueño, clave, límites, pausa y reconciliación), el completado de cultivos anteriores, la caché con respaldo local y la



cadena de seguridad de los controladores. El E2E central de [.github](#) recorre la API completa con 35 comprobaciones.

SECCIÓN 09

Operación

TAREA	CÓMO
Estado	GET /health → base, broker, caché, IA y simulador (503 si la base cae)
Logs	docker logs -f smartpot-api
Imagen	ghcr.io/smartpottech/smartpot-api: compila en una etapa aparte, usuario 1000, solo lectura con tmpfs en /tmp y HEALTHCHECK
Despliegue	Cada cambio en main pasa por el CI, publica la imagen y pide el despliegue central de .github
Rotar una clave	Pestaña Dispositivo de la PWA o POST /api/v1/crops/{id}/device/key (solo reales)